

universidade de aveiro

Vigia-Pirata (vp)

João Reis 132982
Rúben Marques 133155

O que o comando tem de detetar?



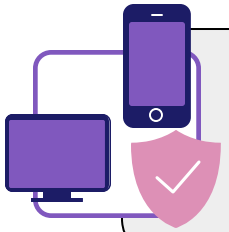
Novos ficheiros



Ficheiros apagados



Alteração de posse



Modificação do tamanho ou conteúdo



Alteração das permissões



Alteração da data de modificação

Inicialização

```
#!/bin/bash

# variaveis que guardam o que o utilizador pede na linha de comandos
modo_p=false
modo_v=false
ficheiro_bd="/tmp/.pdb" #por defeito a bd fica neste caminho
padrao_ignorar=""
usar_logger=false
dir=() #array com os diretorios a proteger

function msg_erro()
{
    echo "Uso: ./vp [{-p <dir1[<dir2>...] | -v}] [-f <ficheiro>] [-e <ignore_pattern>] [-s]"
    echo "       -p : Protege e guarda informação acerca de um ou mais diretórios"
    echo "       -v : Verifica alterações em ficheiros desde que foram protegidos"
    echo "       -f : Indicação do nome do ficheiro onde a base de dados será guardada (Por defeito será tmp/.pdb)"
    echo "       -e : Permite a exclusão de ficheiros voláteis (ex. *.tmp, *.log, /tmp/*) da proteção e verificação"
    echo "       -s : Emissão de alertas de alta prioridade para o sistema de logging usando o logger"
}
```



Leitura de argumentos

```
while [ $# -gt 0 ]; do

    # o getopts processa todas as flags seguidas até encontrar um argumento sem "-"
    while getopts ":pvf:e:s" opt; do
        case "$opt" in
            p) modo_p=true ;;
            v) modo_v=true ;;
            f) ficheiro_bd="$OPTARG" ;;
            e) padrao_ignorar="$OPTARG" ;;
            s) usar_logger=true ;;
            \?) echo "Invalid switch -$OPTARG"; msg_erro; exit 1 ;;
            :)  echo "Falta argumento para -$OPTARG"; msg_erro; exit 1 ;;
        esac
    done

    # elimina da linha de comandos as flags que o getopts acabou de ler
    shift $((OPTIND - 1))

    # se ainda sobrar algum argumento, significa que o getopts parou num diretório (argumento de -p)
    if [ $# -gt 0 ]; then
        dir+=("$1") # guarda o diretório no array (argumento de -p)
        shift              # avança 1 argumento para "consumir" o diretório (argumento de -p)
        OPTIND=1           # reinicia o índice do getopts para ele voltar a procurar flags
    fi
done
```



Recolha de metadados

```
function calcular_perm(){ #r=4;w=2;x=1;-=0
    local p="${1:1:9}" # ${variavel:inicio:tamanho}, a posição 0 não interessa pois apenas indica o tipo de ficheiro
    local numero="" i grupo valor #cria 4 variaveis, numero que é o resultado inicializado com ""
                                  #i é a posição onde começa cada grupo, grupo sao as 3 letras e o valor é o valor do grupo (rwx)

    for i in 0 3 6; do #ciclo acontece 3 vezes
        grupo=${p:$i:3}
        valor=0

        if [ "${grupo:0:1}" != "-" ]; then #permissao tem a ordem rwx
            valor=$((valor + 4))
        fi
        if [ "${grupo:1:1}" != "-" ]; then
            valor=$((valor + 2))
        fi
        if [ "${grupo:2:1}" != "-" ]; then
            valor=$((valor + 1))
        fi

        numero="$numero$valor" #junta o textl de numero e valor. ex: ""+"7"
    done

    echo "$numero"
}
```



Exclusão com -e

```
1  function excluir() {
2      if [ -n "$padrao_ignorar" ]; then # se -e foi chamado
3          case "$1" in
4              $padrao_ignorar) return 0 ;; # 0 = Verdadeiro (coincide com o padrão, é para excluir)
5              *) return 1 ;;               # 1 = Falso (não coincide)
6          esac
7      fi
8      return 1;
9  }
```



Modo de proteção

```
function proteger() {
    if [ ${#dir[@]} -eq 0 ]; then  # se o numero de diretorios for igual a 0
        echo "Tem de ser indicado algum diretorio."
        msg_erro
        exit 1
    fi

    rm -f "$ficheiro_bd" # é preciso remover a bd antiga pois ao fim de ser criada só é possível ler e nao editar, ou seja nao é possivel executar duas vezes -p na mema BD
    if ! : > "$ficheiro_bd"; then # se nao for possivel criar uma bd, se for cria uma vazia
        echo "Não foi possível criar a base de dados."
        exit 1
    fi

    # guarda na BD os nomes dos diretorios protegidos (linhas #DIR|nome)
    local diretorios_protegidos=()

    for diretorio in "${dir[@]}"; do
        if [ -d "$diretorio" ]; then
            echo "#DIR|$diretorio" >> "$ficheiro_bd"
            diretorios_protegidos+=("$diretorio")
        else
            echo "O diretório '$diretorio' não existe."
        fi
    done
```



Modo de proteção

```
ficheiros=0 # iniciar contador de numero de ficheiros

    for diretorio in "${diretorios_protegidos[@]}"; do # percorre todos os elementos do array

        while IFS= read -r ficheiro; do # IFS evita que o read apague espaços no inicio/fim da linha e -r evita que \ seja interpretado como caracter especial

                if excluir "$ficheiro"; then
                    continue # se for para excluir salta para o outro ficheiro
                fi

                metadados=$(obter_metadados "$ficheiro") # guarda as informações em metadados
                echo "$ficheiro|$metadados" >> "$ficheiro_bd" #escreve o caminho e os metadados na bd
                ficheiros=$((ficheiros + 1))


        done < <(find "$diretorio" -type f)
    done

    chmod 400 "$ficheiro_bd" # deixa a bd so com leitura para o dono ("acesso READ_OLNY")
    echo "Base de dados de proteção $ficheiro_bd criada com sucesso."
    echo "Ficheiros processados: $ficheiros | Diretórios protegidos: ${diretorios_protegidos[*]}"
}
```



Verificação

```bash
function verificar() {

    if [ ! -f "$ficheiro_bd" ]; then
        echo "A base de dados '$ficheiro_bd' não existe, tem de ser criada com a flag -p."
        msg_erro
        exit 1 # termina com erro
    fi

    local verificados=0 modificados=0 apagados=0 novos=0 # variaveis que so existem dentro da função logo usamos local
    local ficheiro guardado atual desc d dif # linha da base de dados lida/ o caminho do ficheiro/ os metadados antigos e os novos
    local dirs_protegidos=()
    local sep="------------------------------------------------------------------------------------------------------------------------------------------------------"
```



Verificação

```
1  if [ "$atual" != "$guardado" ]; then # se forem diferentes isto vai acontecer
2              # um campo por linha, para o diff mostrar só os que mudaram
3              dif=$(diff <(tr ',' '\n' <<< "$guardado") <(tr ',' '\n' <<< "$atual"))
4
5              desc=""
6              grep -qE "USER|GROUP" <<< "$dif" && desc="$desc Posse"
7              grep -q "PERM" <<< "$dif" && desc="$desc Permissões"
8              grep -qE "SIZE|SHA256" <<< "$dif" && desc="$desc Conteúdo"
9              grep -q "MTIME" <<< "$dif" && desc="$desc Data_de_modificação" # com _ para não ser partido pelo "e" abaixo
10             desc=${desc# }
11             desc=${desc// / e }
12             desc=${desc//_/ } # volta a pôr espaços em "Data de modificação"
13
14             alerta "$ficheiro foi MODIFICADO (Alteração de $desc)"
15             echo "  Informação inicial: $guardado"
16             echo "  Informação atual:  $atual"
17             echo "$sep"
18             modificados=$((modificados + 1))
19         fi
20     done < "$ficheiro_bd" #vamos buscar as linhas que estamos a ler a bd
```



Verificação

```
for d in "${dirs_protegidos[@]}"; do # percorre os elementos do array
        while IFS= read -r ficheiro; do
            if excluir "$ficheiro"; then
                continue
            fi # salta os excluidos

            if ! grep -qF "$ficheiro|" "$ficheiro_bd"; then # -q responde se foi enocntrado ou nao mas nao imprime nada e o -F diferfencia os caracteres especiais
               #se nao tiver na bd acontece ito // procura o caminho na bd
                alerta "$ficheiro é NOVO"
                echo "  Informação atual:  $(obter_metadados "$ficheiro")"
                echo "$sep"
                novos=$((novos + 1))
            fi
        done < <(find "$d" -type f) # procura dentro do diretorio d e os subdiretorios os ficheiros apenas file (f)
    done


    echo "Resumo da Verificação: $verificados verificados | $modificados Modificado | $apagados Apagado | $novos Novo"

    [ $((modificados + apagados + novos)) -eq 0 ]
}
```



Alertas com -s

```
1  function alerta() {
2      echo "$1"
3      if [ "$usar_logger" = true ]; then
4          logger -p user.alert -t vp "$1" # -p defininimos o nivel de prioridade o -t é para colcoar o programa vp a gerar o log
5      fi
6  }
```



Restante código

```
if [ "$modo_p" = true ] && [ "$modo_v" = true ]; then # se for introduzido -p e -v nao pode
    echo "Não é possível introduzir -p e -v simultaneamente."
    msg_erro
    exit 1
fi

if [ "$modo_p" = false ] && [ "$modo_v" = false ]; then # se não for introduzido nem -p nem -v automaticamente é implementado o a flag -v
    modo_v=true
f
```

```
if [ "$modo_p" = true ]; then
    proteger
else
    verificar
    exit $?
fi
```



TESTES

Dificuldades e Aprendizagens

- Aprendizagem de novos comandos Bash.
- Combinação de funções para resultados específicos.
- Resolução de problemas e erros no código.
- Trabalho em equipa e divisão de tarefas.
- Criação de um README para documentar a evolução do código.
- Organização e integração das diferentes funcionalidades.





Obrigado!